Vitrek V71 HiPot Controller

A Python/Flask web application for controlling the **Vitrek V71 Hi-Pot Tester** over USB (HID-to-UART) or RS-232. Runs test sequences, stores results in a local SQLite database, and exports to Excel for SharePoint sync. Built at Juniper Design.

> **Print-ready PDF:** `docs/pdf/README.pdf`

Features

- **USB and RS-232 support** — USB via Silicon Labs CP2110 HID-to-UART DLL (SLABHIDtoUART.dll); RS-232 via pyserial
- **Full V7X command set** — ACW, DCW, IR, GB, CONT test steps; sequence management; live measurements
- **Web UI** — browser-based interface, no installation needed on operator machines (just open `http://localhost:5000`)
- **SQLite results database** — every test session and step result persisted locally
- **Excel export** — formatted, color-coded workbook per session or bulk; ready for SharePoint

Quick Start

1. Install dependencies

```
pip install -r requirements.txt
```

2. Connect the V71

- **USB**: Set `CONFIG MENU → INTERFACE = USB` on the V71 front panel. Connect the USB-B cable.
- **RS-232**: Set `CONFIG MENU → INTERFACE = RS232` and matching baud rate. Use a 9-wire null-modem cable.

3. Run the app

```
python app.py
```

Open `http://localhost:5000` in your browser.

Project Structure

```
├── app.py              Flask web app + REST API + embedded UI
├── v71_driver.py       Low-level USB/serial driver (ctypes + pyserial)
├── database.py         SQLite schema and CRUD
├── excel_export.py     openpyxl Excel workbook generator
├── requirements.txt
├── hipot_results.db    SQLite database (auto-created on first run)
└── docs/
    └── V7x_Series_Operating_Manual.pdf
```

REST API

Method	Endpoint	Description
POST	`/api/connect`	Connect to V71 (`{"mode":"usb"}` or `{"mode":"serial","port":"COM3","baud":115200}`)
POST	`/api/disconnect`	Disconnect
GET	`/api/status`	Current connection + run state
POST	`/api/run`	Start a test sequence (see below)
POST	`/api/abort`	Abort running test
POST	`/api/cont`	Continue from HOLD step
GET	`/api/live`	Live VOLTS/AMPS/OHMS during a run
GET	`/api/sessions`	Recent test sessions
GET	`/api/session/<id>`	Session detail + step results
GET	`/api/export`	Download all sessions as Excel
GET	`/api/export/<id>`	Download single session as Excel

Example: run a test via API

```
POST /api/run
{
  "operator": "Jason",
  "part_number": "PCB-001",
  "serial_number": "SN-12345",
  "steps": [
    { "type": "ACW", "voltage": 1500, "ramp": 2, "dwell": 60, "max_leakage": 0.005 },
    { "type": "IR",  "voltage": 500,  "dwell": 60, "min_resistance": 100000000 },
    { "type": "GB",  "current": 25,   "dwell": 5,  "max_ohm": 0.1 }
  ]
}
```

USB Communication Details

The V71 uses a **Silicon Labs CP2110 HID-to-UART bridge**. Communication is handled through:

- `SLABHIDtoUART.dll` + `SLABHIDDevice.dll` (x64 versions from `software/drivers/`)
- USB VID: `4292` (0x10C4), PID: `34869` (0x8835)
- UART config: 115200 baud, 8N1, RTS/CTS flow control
- Protocol: ASCII commands terminated with `\r\n`

Key commands (from Section 6 of the operating manual):

Command	Description
`*IDN?`	Identify device
`*RST`	Reset / clear
`*ERR?`	Read error register
`NOSEQ`	Clear active sequence
`ADD,ACW,...`	Add ACW test step
`ADD,DCW,...`	Add DCW test step
`ADD,IR,...`	Add IR test step
`ADD,GB,...`	Add Ground Bond step
`ADD,CONT,...`	Add Continuity step
`RUN`	Start active sequence
`ABORT`	Abort running sequence
`RUN?`	Query if running (0/1)
`RSLT?`	Overall result bitmask
`STAT?`	Per-step pass/fail string
`STEPRSLT?,<n>`	Detailed results for step n
`MEASRSLT?,AMPS`	Live current measurement
`MEASRSLT?,VOLTS`	Live voltage measurement

Excel Export

Each export workbook contains: - **Summary sheet** — all sessions with pass/fail, sortable/filterable - **Per-session sheets** — full step results with color coding (green=pass, red=fail)

The `.xlsx` file is suitable for direct upload or auto-sync to SharePoint.

Notes

- The DLL must be accessible at `software/drivers/USB_DLLs_and_Headers/USB DLLs and Headers/x64/`
- The app must be compiled/run as **x64** to match the provided DLL architecture
- After a DUT breakdown event on USB, the V71 may disconnect and reconnect — the driver detects this and surfaces an error; simply reconnect from the UI
- RS-232 requires hardware handshaking (RTS/CTS) — ensure your cable is fully wired (9-wire null modem)